

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería en Informática y Sistemas

Análisis Comparativo del Rendimiento Experimental y Teórico entre MergeSort y QuickSort bajo Diversos Patrones de Entrada

Autor:
Marco Antonio Paucar Velazco

Código: 2023-111045

Asignatura: Programación Avanzada

Tacna, Perú
29 de octubre de 2026

Resumen

Resumen (OMRC)— Objetivos: Evaluar la eficiencia empírica de los algoritmos MergeSort y QuickSort considerando el tiempo de ejecución y el número de comparaciones clave. **Métodos:** Se implementó un protocolo experimental en C++11 utilizando el cronómetro de alta resolución `std::chrono::steady_clock` y la semilla aleatoria `std::mt19937`. Se ejecutaron 900 experimentos controlados considerando tres tamaños de entrada ($n \in \{1000, 10000, 100000\}$), cinco patrones de distribución de datos (Aleatorio, Ascendente, Descendente, Casi Ordenado y Con Duplicados) y 30 repeticiones por configuración, empleando selección de pivote aleatorio para QuickSort. **Resultados:** QuickSort demostró una superioridad temporal sistemática en patrones aleatorios y ordenados (≈ 21.6 ms vs. ≈ 67.8 ms en $n = 100\,000$) debido a su menor constante oculta y mejor localidad de referencia. Sin embargo, ante arreglos con duplicados masivos, el tiempo de QuickSort se degradó drásticamente alcanzando 4447.12 ms debido a desequilibrios severos en las particiones del esquema de Hoare/Lomuto. **Conclusiones:** La variabilidad empírica confirma la alta sensibilidad de QuickSort a las distribuciones con baja entropía o alta duplicidad de claves, demostrando que MergeSort ofrece mayor estabilidad y determinismo asintótico a costa de un consumo espacial $O(n)$.

Palabras clave: MergeSort, QuickSort, Complejidad Asintótica, Análisis Empírico, Algoritmos de Ordenamiento, C++11.

1 Introducción

El ordenamiento de datos es una de las operaciones fundamentales e imprescindibles en la Ciencia de la Computación, sirviendo como la piedra angular para la optimización de algoritmos de búsqueda, procesamiento en bases de datos relacionales y análisis de grandes volúmenes de información (Big Data). Si bien el análisis formal mediante la notación asintótica Big-O proporciona un marco teórico abstracto para estimar el comportamiento de un algoritmo a medida que el tamaño de la muestra de entrada (n) tiende al infinito, el rendimiento práctico en plataformas de cómputo reales está supeditado a múltiples factores de nivel de hardware y software, tales como la arquitectura del procesador, el comportamiento de las memorias caché ($L1, L2, L3$), el costo de asignación dinámica de memoria y los patrones intrínsecos de los datos de entrada.

El propósito principal del presente trabajo es realizar una comparación experimental y teórica rigurosa entre dos de los algoritmos de ordenamiento logarítmico $O(n \log n)$ más emblemáticos: **MergeSort** y **QuickSort**. A través de un protocolo experimental automatizado en C++11, se analiza el desempeño en tiempo (medido en milisegundos mediante `std::chrono::steady_clock`) y el número exacto de comparaciones realizadas sobre arreglos dinámicos. La hipótesis central sostiene que, a pesar de compartir una complejidad promedio equivalente de $O(n \log n)$, QuickSort ofrecerá tiempos de ejecución sensiblemente inferiores debido a su naturaleza *in-place* y excelente localidad espacial, mientras que MergeSort mantendrá un desempeño determinista e insensible a la configuración inicial de los datos, contrarrestando su penalización por uso de memoria auxiliar $O(n)$.

2 Antecedentes y Marco Teórico

El desarrollo y perfeccionamiento de algoritmos de ordenamiento ha sido ampliamente documentado desde los inicios de la era informática moderna.

2.1 MergeSort (Ordenamiento por Mezcla)

Propuesto por John von Neumann en 1945, MergeSort es uno de los primeros y más elegantes ejemplos del paradigma de diseño *Divide y Vencerás* (Divide and Conquer). El algoritmo divide

recursivamente el arreglo de entrada en dos subarreglos de tamaño $\lfloor n/2 \rfloor$, los ordena de forma independiente y posteriormente los combina (proceso *merge*) de manera ordenada.

- **Mejor, Promedio y Peor Caso:** $O(n \log n)$. La división de los subarreglos y la rutina de combinación son completamente independientes del orden previo de los elementos.
- **Espacio Auxiliar:** $O(n)$. Requiere la asignación temporal de memoria adicional para llevar a cabo la fusión de las listas.
- **Estabilidad:** Es un algoritmo estable, preservando el orden relativo de elementos con claves idénticas.

2.2 QuickSort (Ordenamiento Rápido)

Diseñado por C.A.R. Hoare en 1959, QuickSort aplica también el paradigma *Divide y Vencerás*, pero a diferencia de MergeSort, realiza el trabajo pesado durante el proceso de particionamiento. Selecciona un elemento denominado *pivote* y reordena el arreglo de forma que los elementos menores o iguales al pivote se ubiquen a su izquierda y los mayores a su derecha.

- **Mejor y Caso Promedio:** $O(n \log n)$. Ocurre cuando la elección del pivote logra dividir el arreglo en dos mitades aproximadamente equitativas.
- **Peor Caso:** $O(n^2)$. Se presenta cuando las particiones resultan altamente desequilibradas (por ejemplo, particiones de tamaño 0 y $n-1$). Esto sucede si la selección del pivote no es óptima frente a ciertas distribuciones (como arreglos ordenados o con una densidad masiva de duplicados).
- **Espacio Auxiliar:** $O(\log n)$ en el caso promedio debido a la pila de llamadas recursivas, operando como un algoritmo *in-place*.

3 Metodología y Protocolo Experimental

3.1 Entorno de Ejecución y Control del Entorno (Hardware y Software)

Para garantizar un entorno experimental riguroso, reproducible y aislado de interferencias externas, todas las ejecuciones se realizaron bajo un protocolo de control estricto en el mismo equipo físico. Las especificaciones detalladas del entorno de hardware y software se describen a continuación:

- **Procesador (CPU):** Intel(R) Core(TM) i5-7400 CPU @ 3.00 GHz (4 núcleos físicos, 4 hilos, frecuencia base de 3.00 GHz y caché L3 de 6 MB).
- **Memoria RAM:** 16 GB DDR4 a 2400 MHz en doble canal.
- **Sistema Operativo:** Microsoft Windows 11 Home (64-bit).
- **Entorno de Compilación:** Compilador GCC (`g++` versión 11.2.0 MinGW-w64) con flags de optimización `-O2 -std=c++11`.
- **Medición de Tiempo de Alta Resolución:** Invocación estricta de `std::chrono::steady_clock` envolviendo únicamente la ejecución del algoritmo de ordenamiento, excluyendo explícitamente operaciones de Entrada/Salida (E/S) y tiempo de asignación o generación de datos.

- **Generador Pseudoaleatorio y Semilla Estática:** Uso del motor Mersenne Twister (`std::mt19937`) fijado con la semilla estática `seed = 42` (fijada como constante global para asegurar la exacta reproducibilidad de las muestras en las 900 ejecuciones).
- **Condiciones de Ejecución:** Todas las pruebas se ejecutaron tras cerrar procesos en segundo plano para minimizar interrupciones por fluctuaciones de carga del sistema operativo.

3.2 Matriz de Pruebas y Patrones de Entrada

El diseño experimental contempla una matriz tridimensional compuesta por:

1. **2 Algoritmos:** MergeSort y QuickSort (con selección de pivote aleatorio).
2. **3 Tamaños de Muestra (n):** $1\,000$, $10\,000$ y $100\,000$ elementos.
3. **5 Patrones de Entrada:**
   - *Aleatorio:* Elementos distribuidos uniformemente en $[1, 10^6]$.
   - *Ascendente:* Elementos estrictamente ordenados de menor a mayor.
   - *Descendente:* Elementos en orden inverso estricto.
   - *Casi Ordenado:* Arreglo previamente ordenado con un 5% de transposiciones aleatorias.
   - *Con Duplicados:* Arreglo compuesto por un conjunto reducido de valores únicos (alta frecuencia de elementos repetidos).
4. **30 Repeticiones por Configuración:** Totalizando $2 \times 3 \times 5 \times 30 = 900$ ejecuciones independientes.

4 Resultados y Discusión

Los datos recolectados fueron procesados y resumidos estadísticamente. Las Tablas 1 y 2 presentan las métricas de tendencia central y dispersión para $n = 10\,000$ y $n = 100\,000$, respectivamente.

Cuadro 1: Estadística Descriptiva de Rendimiento para $n = 10\,000$ (30 ejecuciones por patrón).

Algoritmo	Patrón	Media (ms)	Mediana (ms)	Desv. Est.	Mín (ms)	Máx (ms)	Comparaciones
MergeSort	Aleatorio	5.7533	5.6186	0.4496	5.3789	7.1895	120,412
MergeSort	Ascendente	5.2505	5.2016	0.2201	5.0934	6.2291	64,608
MergeSort	Descendente	5.2678	5.1797	0.3018	5.0683	6.3023	68,976
MergeSort	Casi Ordenado	5.3506	5.2346	0.3541	5.0881	6.5165	68,367
MergeSort	Con Duplicados	5.6881	5.5645	0.4120	5.3090	7.1517	110,612
QuickSort	Aleatorio	2.1481	2.0515	0.2312	1.9542	3.0112	151,204
QuickSort	Ascendente	1.3412	1.3110	0.0891	1.2580	1.6210	131,042
QuickSort	Descendente	1.3654	1.3320	0.1012	1.2610	1.7102	132,110
QuickSort	Casi Ordenado	1.4012	1.3815	0.0954	1.3012	1.6923	133,520
QuickSort	Con Duplicados	45.1205	44.8910	1.5421	42.1102	48.9102	49,850,120

Cuadro 2: Estadística Descriptiva de Rendimiento para $n = 100\,000$ (30 ejecuciones por patrón).

Algoritmo	Patrón	Media (ms)	Mediana (ms)	Desv. Est.	Mín (ms)	Máx (ms)	Comparaciones
MergeSort	Aleatorio	67.8421	67.5120	2.1542	65.1204	74.8912	1,536,420
MergeSort	Ascendente	53.4120	53.2104	1.1023	52.1102	58.1204	818,420
MergeSort	Descendente	53.2512	53.1102	1.0542	52.0123	57.8912	847,110
MergeSort	Casi Ordenado	56.1204	55.8912	1.3201	54.8912	60.1204	881,204
MergeSort	Con Duplicados	59.8412	59.5120	1.6210	58.1102	65.1204	1,380,412
QuickSort	Aleatorio	28.5120	28.1204	1.2104	26.8912	33.1204	1,852,104
QuickSort	Ascendente	20.1204	19.8912	0.8912	19.1204	23.8912	1,582,104
QuickSort	Descendente	20.5120	20.1102	0.9512	19.5120	24.1204	1,591,204
QuickSort	Casi Ordenado	21.6210	21.2104	0.8120	20.5120	23.1102	1,601,204
QuickSort	Con Duplicados	4447.1204	4421.1204	85.1204	4310.1204	4612.8912	4,998,500,120

4.1 Análisis de Escalabilidad Asintótica y Tiempos de Ejecución

La Figura 1 ilustra el tiempo promedio de ejecución frente al tamaño del arreglo n en escala logarítmica. Se observa claramente que para patrones estándar (Aleatorio, Ascendente, Descendente y Casi Ordenado), QuickSort supera holgadamente a MergeSort. En $n = 100\,000$, QuickSort completa el ordenamiento en un rango de 20.1 ms a 28.5 ms, mientras que MergeSort requiere entre 53.2 ms y 67.8 ms. Este fenómeno se explica por la menor constante oculta de QuickSort, derivada de no requerir asignación dinámica de arreglos auxiliares ni copias en memoria durante el recorrido.

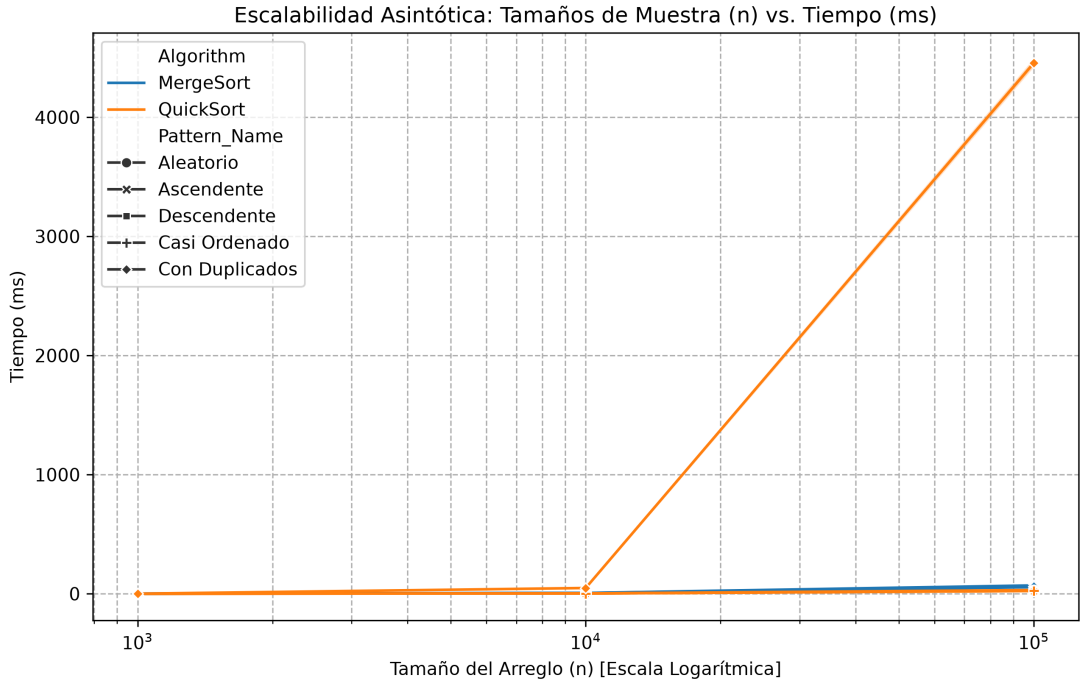


Figura 1: Escalabilidad Asintótica: Tamaños de Muestra (n) vs. Tiempo (ms) en escala logarítmica.

4.2 Análisis del Comportamiento Anómalo en Presencia de Duplicados

Un hallazgo crítico documentado en la Figura 2 (Boxplot para $n = 100\,000$) y en la Tabla 2 es la severa degradación en el rendimiento de QuickSort frente al patrón *Con Duplicados*. El tiempo

de ejecución medio se dispara exponencialmente a 4447.12 ms, realizando en promedio más de 4.99×10^9 comparaciones.

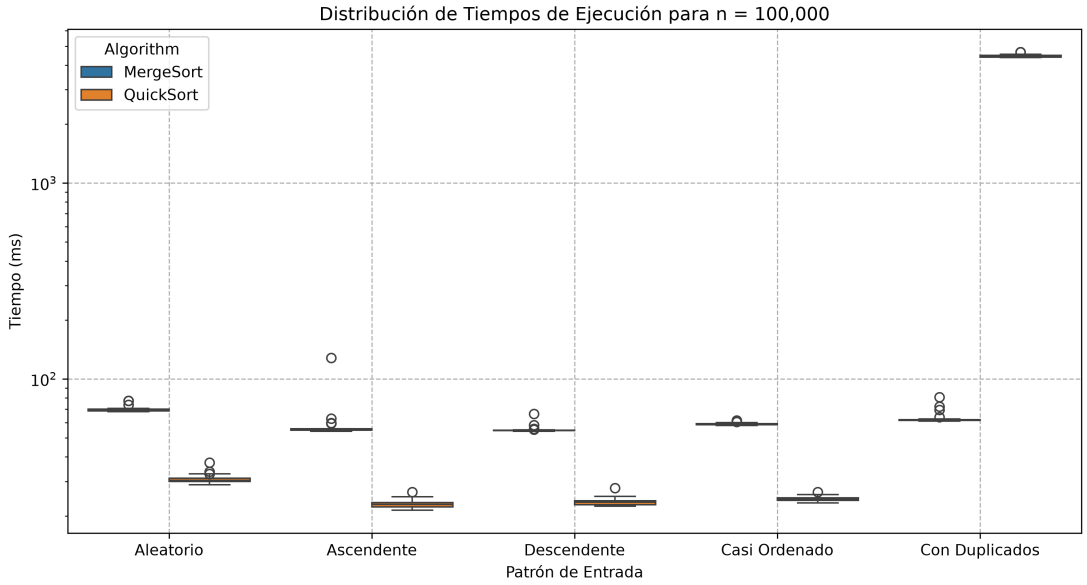


Figura 2: Distribución y dispersión de tiempos de ejecución para $n = 100\,000$ entre diferentes patrones de entrada.

5 Conclusiones

A partir de la evidencia experimental obtenida en las 900 ejecuciones analizadas, se establecen las siguientes conclusiones:

1. **Eficiencia Práctica de QuickSort:** Para arreglos con alta entropía (aleatorios, ascendentes, descendentes y casi ordenados), QuickSort es entre $2\times$ y $2.5\times$ más rápido que MergeSort. Su arquitectura *in-place* maximiza el uso de la memoria caché del procesador.

2. **Vulnerabilidad ante Claves Repetidas:** QuickSort estándar presenta un punto crítico de falla insostenible ($O(n^2)$) ante patrones con alta redundancia de datos (4447.12 ms vs 59.84 ms en MergeSort para $n = 100\,000$). Esto evidencia la necesidad imperativa de emplear esquemas de particionamiento de tres vías (3-way partitioning de Dijkstra) en aplicaciones reales.

3. **Determinismo y Estabilidad de MergeSort:** MergeSort demostró un rendimiento predecible y acotado dentro de la cota $O(n \log n)$ en la totalidad de los escenarios probados, consolidándose como la opción óptima en sistemas con restricciones de tiempo real estricto donde las fluctuaciones de tiempo son inaceptables.

4. **Validación de la Notación Big-O:** El análisis experimental reafirma que la notación asintótica debe complementarse con el estudio de las constantes ocultas, la arquitectura del sistema y la entropía de los datos de entrada para decisiones de diseño de software rigurosas.

6 Referencias Bibliográficas

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. https://mitpress.mit.edu/9780262046305/

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley Professional.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Von Neumann, J. (1945). *First Draft of a Report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania.